

El LocalStorage Tema Bonus

El localStorage es una API de almacenamiento en el navegador que permite guardar pares clave-valor de forma persistente. Los datos se mantienen incluso después de cerrar el navegador, haciéndolo ideal para conservar configuraciones, carritos de compras y más.

Características Principales

Persistencia: Los datos no caducan automáticamente; permanecen almacenados hasta que se eliminan manualmente o mediante código..
Alcance local: Los datos solo son accesibles dentro del dominio donde se guardaron..
Capacidad limitada: Aproximadamente 5 MB por dominio..
Solo texto: Los datos almacenados deben ser cadenas de texto. Otros tipos de datos deben serializarse, por ejemplo, con `JSON.stringify()`.

Métodos Comunes

1. Guardar Datos: `setItem(key, value)`

Guarda un valor asociado a una clave.

```
javascriptCopy codelocalStorage.setItem('nombre', 'Carlos');
```

2. Recuperar Datos: `getItem(key)`

Obtiene el valor almacenado en localStorage usando su clave.

```
javascriptCopy codeconst nombre = localStorage.getItem('nombre');  
console.log(nombre); // "Carlos"
```

3. Eliminar Datos Específicos: `removeItem(key)`

Elimina el dato asociado a una clave específica.

```
javascriptCopy codelocalStorage.removeItem('nombre');
```

4. Eliminar Todos los Datos: `clear()`

Borra todo el contenido almacenado en localStorage.

```
javascriptCopy codelocalStorage.clear();
```

5. Obtener Clave por Índice: `key(index)`

Devuelve la clave almacenada en un índice específico.

```
javascriptCopy codeconst clave = localStorage.key(0);  
console.log(clave); // Por ejemplo: "nombre"
```

Ejemplos Prácticos

1. Guardar y Recuperar Datos

```
javascriptCopy code// Guardar datos
```

```
localStorage.setItem('nombre', 'Carlos');
```

```
localStorage.setItem('edad', '30');
```

```
// Recuperar datos
```

```
console.log(localStorage.getItem('nombre')); // "Carlos"
```

```
console.log(localStorage.getItem('edad')); // "30"
```

2. Almacenar Objetos

```
javascriptCopy code// Guardar un objeto
```

```
const usuario = { nombre: 'Carlos', edad: 30 };
```

```
localStorage.setItem('usuario', JSON.stringify(usuario));
```

```
// Recuperar el objeto
```

```
const datosUsuario = JSON.parse(localStorage.getItem('usuario'));
```

```
console.log(datosUsuario.nombre); // "Carlos"
```

```
console.log(datosUsuario.edad); // 30
```

3. Eliminar Datos

```
javascriptCopy code// Guardar y eliminar un dato
```

```
localStorage.setItem('tema', 'oscuro');
```

```
console.log(localStorage.getItem('tema')); // "oscuro"
```

```
localStorage.removeItem('tema');
```

```
console.log(localStorage.getItem('tema')); // null
```

4. Limpiar Todo

javascriptCopy code// Guardar múltiples datos

```
localStorage.setItem('nombre', 'Carlos');
```

```
localStorage.setItem('edad', '30');
```

```
console.log(localStorage.length); // 2
```

// Eliminar todo

```
localStorage.clear();
```

```
console.log(localStorage.length); // 0
```

5. Iterar Sobre Claves y Valores

javascriptCopy code// Guardar datos

```
localStorage.setItem('nombre', 'Carlos');
```

```
localStorage.setItem('edad', '30');
```

// Iterar sobre los datos

```
for (let i = 0; i < localStorage.length; i++) {
```

```
    const clave = localStorage.key(i);
```

```
    console.log(` ${clave}: ${localStorage.getItem(clave)} `);
```

```
}
```

// Salida:

// nombre: Carlos

// edad: 30

6. Manejar Datos Que No Existen

javascriptCopy codeconst valor = localStorage.getItem('noExiste');

```
console.log(valor); // null
```

`localStorage.removeItem('noExiste');` // *No genera error*

Casos de Uso Comunes

1. **1.Configuraciones de Usuario:** Guardar preferencias como tema oscuro, idioma, etc.
2. **2.Carritos de Compra:** Persistir productos seleccionados en un carrito.
3. **3.Datos Temporales:** Recuperar datos de formularios o estados de la aplicación después de recargar la página.

Limitaciones y Precauciones

- Capacidad Limitada: Aproximadamente 5 MB por dominio.
- Solo Texto: Todos los datos deben ser convertidos a texto (por ejemplo, JSON).
- Seguridad: No almacenes datos sensibles como contraseñas, ya que localStorage no está cifrado.
- Alcance del Dominio: Los datos solo son accesibles desde el dominio donde se guardaron.

Cómo Acceder a localStorage en el Navegador

Paso 1: Abrir las Herramientas de Desarrollo

- En cualquier navegador moderno (Chrome, Firefox, Edge, etc.), abre las herramientas de desarrollo. En Windows/Linux:
- Pulsa Ctrl + Shift + I.
- En Mac: Pulsa Cmd + Option + I.
- O haz clic derecho en cualquier parte de la página y selecciona "Inspeccionar".
- En Windows/Linux: Pulsa Ctrl + Shift + I.
- En Mac: Pulsa Cmd + Option + I.
- O haz clic derecho en cualquier parte de la página y selecciona "Inspeccionar".

Paso 2: Navegar a la Sección de localStorage

1. **1.Ve a la pestaña "Aplicación"** (o "Application" en inglés) en las herramientas de desarrollo.

2. **2.**En el menú de la izquierda, busca la sección "Almacenamiento" o "Storage".Dentro de esa sección, selecciona "localStorage".
3. **3.**Dentro de esa sección, selecciona "localStorage".

Paso 3: Visualizar el Contenido

1. **1.**Una vez en la sección de localStorage, verás una lista de todas las claves y valores almacenados para el dominio actual.
2. **2.**La lista tiene dos columnas principales:Key: Las claves que has utilizado al guardar datos.Value: Los valores asociados a esas claves.
3. **3.**Key: Las claves que has utilizado al guardar datos.
4. **4.**Value: Los valores asociados a esas claves.